
SelfBudgeter: Adaptive Token Allocation for Efficient LLM Reasoning

Zheng Li, Qingxiu Dong, Jingyuan Ma, Di Zhang, Zhifang Sui

National Key Laboratory for
Multimedia Information Processing,
School of Computer Science,
Peking University
{lizheng2001, szf}@pku.edu.cn
{dqx, mjj, zhangdi}@stu.pku.edu.cn

Abstract

Recently, large reasoning models demonstrate exceptional performance on various tasks. However, reasoning models inefficiently over-process both trivial and complex queries, leading to resource waste and prolonged user latency. To address this challenge, we propose SelfBudgeter - a self-adaptive controllable reasoning strategy for efficient reasoning. Our approach adopts a dual-phase training paradigm: first, the model learns to pre-estimate the reasoning cost based on the difficulty of the query. Then, we introduce budget-guided GPRO for reinforcement learning, which effectively maintains accuracy while reducing output length. SelfBudgeter allows users to anticipate generation time and make informed decisions about continuing or interrupting the process. Furthermore, our method enables direct manipulation of reasoning length via pre-filling token budget. Experimental results demonstrate that SelfBudgeter can rationally allocate budgets according to problem complexity, achieving up to 74.47% response length compression on the MATH benchmark while maintaining nearly undiminished accuracy.

1 Introduction

Recent large reasoning models, such as O1 [25], has shown remarkable performance in various complex reasoning tasks [11, 27]. The primary success factor lies in the excessively long chain of thought (CoT) process learned through reinforcement learning (RL), which allows the model to break down reasoning steps and scaling test-time compute [30, 19].

However, reasoning models tend to use overly long thought processes even for simple questions. This “overthinking” phenomenon leads to a waste of computational resources and excessive user waiting times [6, 31]. For example, when answering the simple questions such as “What is the answer of 2+3?”, the QwQ-32B model provides 13 different solutions and generates 100 times more tokens than Qwen2.5-72B-Instruct model [28]. Prior studies have explored various approaches to mitigate overthinking through response length control and computation routing. Existing methods mainly include: (1) Prompt-based approaches [15, 36, 23] that implicitly guide length through instructions, (2) Integrated training strategies that teach models to adaptively determine reasoning steps via SFT [22, 20] or RL with length penalties [1, 2, 18], and (3) Router-based [3, 9, 8] architectures employing classifiers to allocate computation paths. While achieving partial progress, these methods either lack precise length control, require additional computational overhead, or fail to explicitly output optimal reasoning lengths [1, 37].

We propose **SelfBudgeter** that enables reasoning models to (1) estimate the minimal token budget required for correct responses when users do not specify token constraints, and (2) generate responses of corresponding lengths while adhering to either self-estimated or user-defined token budgets. SelfBudgeter aims to mitigate the overthinking issue by predicting the minimal possible token budget, thereby significantly reducing user waiting time. As shown in Figure 1, SelfBudgeter can provide a relatively accurate token budget estimation before generating responses, users can precisely anticipate the waiting time and decide whether to wait for the full output or terminate early based on their needs. Additionally, when specific requirements arise, users can pre-fill the token budget field to constrain the model’s response within the given limit, thereby improving interaction efficiency.

Our training framework consists of two main stages. During the Cold-Start stage, we fine-tune the model to learn how to first output its estimated token budget within `<budget>` tags. Subsequently, in the RL training stage, we optimize SelfBudgeter using the GRPO algorithm. For this stage, we design a reward function that primarily focuses on three key aspects: (1) answer correctness, (2) minimal achievable token budget, and (3) consistency between response length and the allocated token budget.

We conduct full-parameter training of Deepseek-R1-Distill-Qwen-1.5B using SelfBudgeter and evaluated its performance on the GSM8K and MATH datasets. Experimental results demonstrate that SelfBudgeter achieves a 74.47% reduction in response length on the MATH dataset while maintaining nearly equivalent accuracy. Furthermore, on the GSM8K dataset, SelfBudgeter simultaneously reduces response length while improving accuracy. Selfbudgeter exhibits excellent capability in predicting output length and, when provided with pre-filled `<budget>` tags, consistently adheres to the specified token budget constraints.

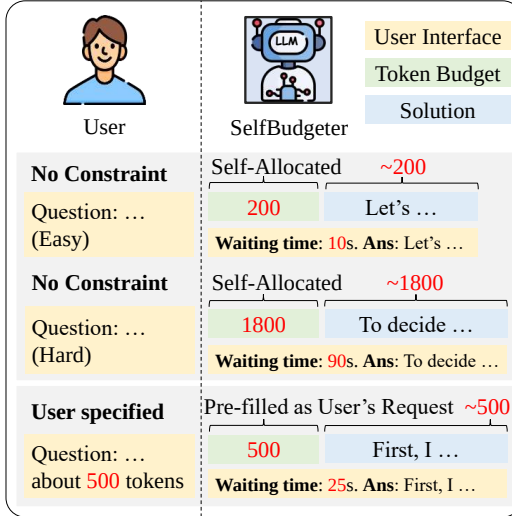


Figure 1: Overview of the SelfBudgeter. SelfBudgeter’s responses comprise two sections: **Token Budget** and **Solution**. For unrestricted queries, it estimates tokens needed for the **Solution** based on problem complexity. When users specify requirements, it pre-fills the **Token Budget** accordingly. The **Solution** generation strictly adheres to **Token Budget** limits, whose numerical value indicates anticipated response duration.

2 Related Work

Overthinking in LLMs The emergence of the reasoning models like O1, Deepseek-R1 and QwQ advanced complex problem-solving through RL-enhanced CoT [33, 25, 11, 27]. However, researchers observed a tendency for reasoning models to overthink simple problems—expending unnecessary computational effort on trivial queries [6, 31]. Excessive long CoT may lead to a decrease in accuracy [34]. Current solutions for overthinking mainly involve following three strategies. Prompt-based methods try to control response length by adding instructions in prompts, but cannot control the length accurately [15, 29, 36, 23]. Integrated Training-based methods try to teach model decide the length by the difficulty of the problems. Supervised fine-tuning(SFT)-based methods collect the dataset with variable length [22, 20, 17, 12, 14, 35, 38]. RL-based methods incorporate length penalties into the reward function [1, 2, 18, 5, 4, 37]. These methods fail to control the length as users’ requirements. And Router-based methods train another model as a classifier [3, 9, 8, 24, 26]. The classifier decide to route the query to fast models or reasoning models. However, an extra classifier means more computation resources are needed. Current methods either sacrifice precise control, require extra computation, or fail to bridge autonomous budget estimation with strict adherence.

Token Budget In addressing the issue of overthinking, a highly intuitive approach involves directly constraining the output length. CCoT [23] attempt to achieve this by incorporating a word budget into the prompt, various approaches—including character, token, and step budgets [15]—have been

attempted by directly incorporating them into prompts, yet achieving precise control over the model’s output behavior remains challenging. TALE [12] introduce, for the first time, the concept of a token budget. TOPS [38] attempt to enable the model to autonomously determine the required effort for solving a given task. However, both TALE and TOPS fail to explicitly guide the model to produce the optimal token budget. They also fail to effectively control the output length according to a given token budget. L1 [1] and Elastic Reasoning [37] can more precisely control the output length under a given token budget, yet they fail to enable the model to autonomously estimate an appropriate response length. Our proposed method enables the model to autonomously estimate the optimal token budget and subsequently generate text in strict adherence to it.

3 Method

To minimize the overthinking problem in LLMs, we propose SelfBudgeter for efficient reasoning. Our method aims to enable the model to autonomously determine an appropriate token budget and generate responses of corresponding length while adhering to this budget. Although reasoning models may occasionally overthink simple problems, their response lengths generally increase with problem difficulty. This phenomenon demonstrates that the model possesses the capability to allocate token quantities reasonably based on problem complexity. Previous works such as L1 [1] and Elastic Reasoning [37] have also demonstrated that models can generate responses of appropriate length according to a given token budget.

Therefore, we design SelfBudgeter, which employs a reward function to guide the model in: (1) learning an output format where it first predicts a token budget before generating the answer, (2) allocating appropriate token budgets based on its own capabilities and question difficulty, and (3) generating solutions with optimal length while ensuring answer accuracy.

3.1 SelfBudgeter

SelfBudgeter is a concise and efficient method for automatic precise length controlled. We design the Precise Budget Control Reward (PreB Reward) to achieve precise control over length. The detailed introduction of PreB Reward can be found in Section 3.3. We employ GRPO algorithm to train the model in predicting appropriate token budgets based on problem difficulty and generating responses with lengths conforming to the specified budget.

Our reward function is formally defined as Formula 1:

$$R(C, F, \ell, b, b_{\max}) = \begin{cases} r_f, & \text{if } F = 0, \\ P_B(b, b_{\max}) + \text{PreB}(s_{\min}^W, s_{\max}^W, \ell, b, \alpha, b_{\text{best}}^W), & \text{if } F = 1 \text{ and } C = 0, \\ P_B(b, b_{\max}) + \text{PreB}(s_{\min}^C, s_{\max}^C, \ell, b, \alpha, b_{\text{best}}^C), & \text{if } F = 1 \text{ and } C = 1. \end{cases} \quad (1)$$

where

$$b_{\text{best}}^C = (1 - \alpha) \cdot b, \quad b_{\text{best}}^W = (1 + \alpha) \cdot b \quad (2)$$

The inputs and hyperparameters in the reward function are listed in Table 1. To ensure stable prediction of the token budget prior to response generation, any responses deviating from the prescribed format will be assigned the minimum reward score of r_f .

Table 1: Input and Hyperparameters (HPs) in the reward function

Input	Description	HPs	Description
C	Correctness for answer	r_f	Penalty for format error
F	Correctness for format	$s_{\min}^{W/C}$	Minimum reward (wrong/correct)
ℓ	Response length	$s_{\max}^{W/C}$	Maximum reward (wrong/correct)
b	Model’s budget	α	Tightness coefficient of budget
$b_{\max}$	Maximum acceptable budget	r_b	Penalty for excessive budget

3.2 Budget Penalty

To enable the model to learn token budget allocation, we introduce a budget penalty module defined by Formula 3. The model incurs a penalty r_b when its estimated token budget exceeds the maximum acceptable budget $b_{\max}$. No penalty is applied when the estimated token budget remains within $b_{\max}$. A detailed introduction of $b_{\max}$ is presented in Section 4.2. Briefly stated, for a given question, $b_{\max}$ equals the response length if the base model can answer it correctly; otherwise, $b_{\max}$ is set to ∞ .

$$P_B(b, b_{\max}) = \begin{cases} 0, & \text{if } b \leq b_{\max}, \\ r_b, & \text{else.} \end{cases} \quad (3)$$

3.3 Precise Budget Control Reward

Inspired by the cosine reward [4], we propose the Precise Budget Control Reward (PreB Reward). While the cosine reward helps mitigate overthinking tendencies, it lacks precise control over output length, as it only constrains the upper bound of the response. To address this limitation, we introduce a tightness coefficient α to better align the response length with the specified token budget.

Given the inherent challenge for models to precisely comply with token budgets, we relax the length constraint to require only approximate adherence within $\alpha \cdot b$ around the target budget b . As shown in Formula 4, when the model’s response length falls outside the specified range, the corresponding reward score plummets to its minimum value $s_{\min}$.

For incorrect responses, the function incentivizes longer reasoning chains (increasing length ℓ) to encourage deeper analysis that might lead to correct conclusions. Conversely, for correct answers, the reward peaks at the minimally sufficient length $(1 - \alpha) \cdot b$ to prevent unnecessary computational overhead while maintaining accuracy. This explains why in Formula 2, the value of b_{best} differs between correct and incorrect responses from the model. This dual mechanism promotes efficient reasoning by adaptively modulating response lengths based on answer correctness.

$$\text{PreB}(s_{\min}, s_{\max}, \ell, b, \alpha, b_{\text{best}}) = \begin{cases} s_{\min}, & \text{if } \frac{|\ell - b|}{b} > \alpha, \\ s_{\min} + (s_{\max} - s_{\min}) \times \frac{1}{2} \left(1 + \cos \left(\pi \cdot \frac{|\ell - b_{\text{best}}|}{2\alpha b} \right) \right), & \text{else.} \end{cases} \quad (4)$$

3.4 Accuracy Reward

To ensure the model’s post-training accuracy does not degrade below its initial performance, we configure hyperparameters to guarantee that the minimum reward for correct responses always exceeds the maximum reward for incorrect responses. Specifically, our design ensures that: A correct response, which has a token budget exceeding $b_{\max}$ and receives the lowest budget following reward $s_{\min}^C$, will yield a higher total reward than an incorrect response that has a token budget within $b_{\max}$ and receives the highest budget following reward $s_{\max}^W$. This constraint is formally expressed as: $s_{\min}^C + r_b \geq s_{\max}^W$.

Overall, the core design of SelfBudgeter consists of three key modules: Budget Penalty, Preb Reward, and Accuracy Reward, which collectively balance length compression, correctness, and precise length control—ultimately delivering a better user experience.

4 Experiment

4.1 Training Template

The existing reasoning models utilize a pair of `<think>`/`</think>` tags to demarcate the thinking process from the final solution output. Building upon this format, we have further incorporated a token budget component.

To enable the model to dynamically allocate token usage based on question difficulty, we design an output format as follows:

<budget>an integer, token budget</budget><solution>response</solution>

The format requires the model to first estimate the required token budget before providing the answer to the question. When no user constraint exists, the model autonomously predicts the token budget. When users specify a token limit, we pre-fill the <budget> field and let the model generate the <solution> within this constraint.

4.2 Data Preprocessing

At this stage, we collect model’s responses to the test questions used in both the cold-start and RL training phases, and then evaluate the correctness and length of the responses.

For the cold-start data, we retain the model’s correct responses along with their lengths and discard incorrect answers to prevent reinforcing the model’s memory of wrong responses.

For the RL training data, we calculate $budget_{\max}$ (for convenience, we will refer to it as $b_{\max}$ in the following sections) using Formula 5, representing the maximum acceptable token budget for a given question. When the model answers correctly, the correctness of the response indicates that the minimum token budget required for a correct answer does not exceed the current length. Therefore, we encourage the model to further compress the response length and set $b_{\max}$ to the current response length. When the model answers incorrectly, the relationship between the minimum token budget needed for correctness and the current length remains unclear, so any token budget is acceptable.

$$b_{\max} = \begin{cases} \text{response length,} & \text{if model answers correctly,} \\ \infty, & \text{else.} \end{cases} \quad (5)$$

4.3 Cold Start

In our actual RL training process, we observe that requiring the model to simultaneously master multiple objectives - learning the new output format, providing appropriate token budgets, generating solutions of corresponding lengths according to the budget, while maintaining or improving accuracy - proved excessively challenging. After extended training periods, the model often only succeeds in adopting the output format without achieving the other goals. Inspired by the Deepseek-R1 training methodology, we introduce a cold-start phase to accelerate training and enable the model to first learn the new output format before proceeding to more complex tasks. The overall training framework is illustrated in Figure 2.

To prevent the model from losing its original reasoning capability during the cold-start phase, fine-tuning must be performed using either the model’s own generated responses or datasets containing long CoT responses. In our approach, we pre-populate the <budget> section with token counts obtained during the preprocessing stage. The <solution> section is filled with the model’s generated responses. And we prepend the instruction prefix shown in Figure 3 to each question.

4.4 Experiment Settings

We conduct experiments on the DeepSeek-R1-Distill-Qwen-1.5B model. We reproduce L1-max using DeepSeek-R1-Distill-Qwen-1.5B. And we select DeepSeek-R1-Distill-Qwen-1.5B and L1-Max as baseline methods for comparative evaluation against SelfBudgeter.

During the cold-start phase, we employ the GSM8K [10] dataset. We retain 5,802 training samples that the model could answer correctly. After preprocessing the training data as described in Sections 4.2 and 4.3, we fine-tune the model. Using this structured dataset, we conduct full-parameter fine-tuning of the model for one epoch. Throughout all data collection and training phases, the model’s temperature parameter is consistently maintained at 0.6.

We also try to employ the s1k-1.1 [21] dataset. This dataset comprises 1,000 mathematically diverse and challenging problems, with solutions generated by Deepseek-R1. The responses from Deepseek-R1 incorporate extensive reasoning chains, ensuring that the model to be trained can maintain its reasoning capabilities while assimilating the output format. Unlike GSM8K, s1k dataset inherently contains high-quality CoT responses. We therefore directly utilize the dataset’s native responses and

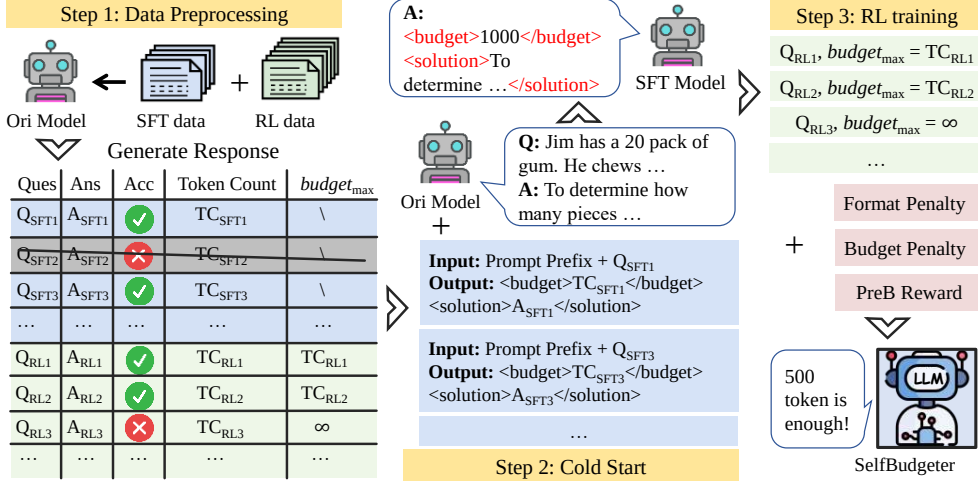


Figure 2: Overview of the SelfBudgeter training framework. The workflow consists of three key steps: (1) **Data preprocessing**: The initial model preprocesses the data to compute token budgets for subsequent training; (2) **Cold-start**: The model is fine-tuned to adopt the new output format; (3) **RL Training**: Through reward functions, the model learns to allocate token budgets and generate compliant outputs.

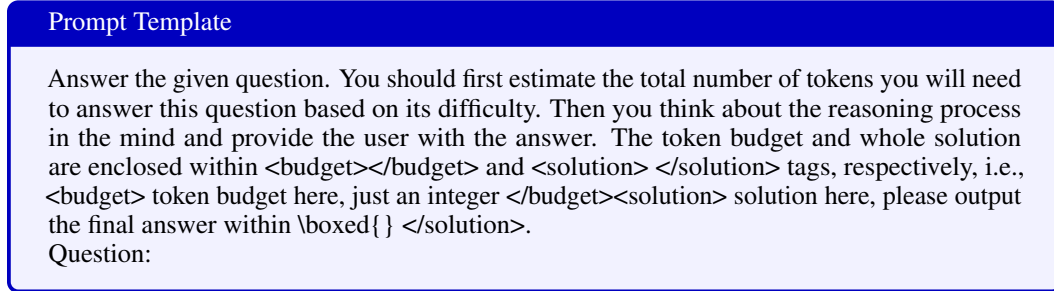


Figure 3: The prompt template used in the cold-start stage.

computed corresponding token counts using the model’s tokenizer to populate our designed template. Following the same protocol, we perform full-parameter fine-tuning for one epoch on this dataset.

During the reinforcement learning phase, we use STILL-3-Preview-RL-Data [7] dataset. It also serves as the training dataset for reproducing L1-max. This dataset collects 30K high-quality samples based on the MATH [13], NuminaMathCoT [16], and AIME 1983-2023 [32] datasets. It includes problems of varying difficulty levels, which also helps the model learn to allocate token counts adaptively based on difficulty. As described in Section 4.2, we compute the maximum acceptable budget (b_{max}) based on the model’s responses, then train the model for 3 epochs on this dataset. More detailed information can be found in Appendix A.

4.5 Main Results

Table 2 presents a comprehensive comparison of model performance on the GSM8K and MATH test sets, evaluated through accuracy (Acc), average response length (Len) and matching degree between token limit and response length (Mat). The table contrasts baseline models with variants of the SelfBudgeter framework, which incorporates different cold-start datasets (GSM or s1k) and varying values of the parameter α (0.2 or 0.5). Given the inherent difficulty for large language models to strictly adhere to length constraints, we define a response as “matched” if its length deviates by $\leq 50\%$ from the token limit, and “unmatched” otherwise. Notably, the token limits for L1 are explicitly specified through prompt templates, whereas SelfBudgeter autonomously estimates its token constraints during inference.

Table 2: Model performance comparison on GSM8K and MATH test sets, showing accuracy (Acc/%), average response length (Len/tokens) and matching rate between token limits and response length (Mat/%). The SelfBudgeter variants with different cold-start data and α parameters are contrasted with baseline models.

Model	GSM8K			MATH		
	Acc $\uparrow$	Len $\downarrow$	Mat $\uparrow$	Acc $\uparrow$	Len $\downarrow$	Mat $\uparrow$
Deepseek-R1-Distill-Qwen-1.5B	78.32	1737.92	-	76.34	3731.25	-
L1-Max(Num Tokens=512)	72.86	568.95	65.35	71.16	1533.39	38.94
L1-Max(Num Tokens=1024)	70.89	742.24	33.36	73.40	1601.45	42.52
L1-Max(Num Tokens=2048)	71.42	636.24	7.66	71.68	1573.22	30.24
*L1-Max(Num Tokens=3600)	74.37	633.50	2.96	77.12	1544.96	17.02
Best *L1 v.s. R1	-3.95	-63.54%	-	+0.78	-58.59%	-
Cold Start (GSM)	71.95	1003.79	85.82	64.74	3043.29	41.16
SelfBudgeter (GSM, $\alpha = 0.2$)	76.27	523.77	97.65	63.46	779.54	95.82
SelfBudgeter (GSM, $\alpha = 0.5$)	74.68	520.82	96.97	63.78	777.80	96.66
Cold Start (s1k)	82.49	1983.29	21.76	76.64	4001.29	23.28
*SelfBudgeter (s1k, $\alpha = 0.2$)	81.50	662.08	70.74	74.18	919.27	78.36
SelfBudgeter (s1k, $\alpha = 0.5$)	80.44	719.36	71.19	72.60	1022.99	79.76
Best *SelfBudgeter v.s. R1	+3.18	-62%	-	-2.16	-74.47%	-

Baseline Comparison While the Deepseek-R1-Distill-Qwen-1.5B baseline achieves strong accuracy, it requires substantially longer responses, particularly on MATH (3245.65 tokens). On the MATH dataset, our method achieves a **75%** reduction in response length (e.g., 919.27 vs. 3731.25 tokens for $\alpha = 0.2$ with s1k initialization) with only a moderate accuracy decrease. Moreover, under this configuration, SelfBudgeter demonstrates strong performance on the GSM8K dataset as well. While exhibiting a marginal increase in response length, it achieves a 3.18% improvement in accuracy (81.50 vs. 78.32) compared to the baseline. SelfBudgeter achieves high accuracy while substantially reducing actual response lengths, demonstrating the effectiveness of its training framework in addressing overthinking problem.

SelfBudgeter not only outperforms L1-Max in efficient reasoning, but also generates responses that better adhere to token limits. On the GSM8K dataset, SelfBudgeter achieves comparable response compression ratios to L1-Max while demonstrating a 7.13% higher accuracy. On the MATH dataset, it maintains similar accuracy to L1-Max but improves the compression ratio by 15.88%. These results substantiate SelfBudgeter’s superiority over L1-Max in efficient reasoning.

Notably, L1-Max demonstrates significantly inferior performance in adhering to token constraints compared to SelfBudgeter. Across four different Num Tokens settings, L1-Max maintains consistent average response lengths of approximately 600 tokens on GSM8K and 1500 tokens on MATH. In stark contrast, SelfBudgeter achieves matching rates (Mat) exceeding 70% for self-imposed token limits, with peak performance reaching 97.65%.

Cold-start Data Selection The choice of initialization data substantially impacts model performance. SelfBudgeters initialized with the **s1k** dataset outperform their GSM-initialized SelfBudgeters by 8.82–10.72 percentage points on MATH (74.18% vs. 63.46% for $\alpha = 0.2$) and 5.23–5.76 percentage points on GSM8K (80.44% vs. 74.68% for $\alpha = 0.5$). While SelfBudgeters with GSM-initialized exhibit lower accuracy, they generate significantly more concise responses compared to s1k-initialized SelfBudgeters. Specifically, GSM-initialized SelfBudgeters reduces response length by approximately 15–24% on MATH and achieves 21–28% length reduction on GSM8K. This performance gap highlights the importance of high-quality initialization for the budgeting mechanism.

As shown in Table 2, significant performance variations exist between models fine-tuned with different cold-start datasets. The s1k-fine-tuned model demonstrates superior accuracy over the GSM-fine-tuned counterpart, achieving 10.54% and 11.90% higher accuracy on GSM8K and MATH respectively. However, this comes at the cost of substantially longer responses, with the s1k model generating 97.58% and 31.48% lengthier outputs on GSM8K and MATH. This discrepancy stems

from the s1k dataset’s responses being generated by Deepseek-R1, which produces higher-quality outputs than those self-generated by Deepseek-R1-Distill-Qwen-1.5B. Additionally, the s1k dataset’s average length of 7,677.43 tokens (we only retained correct responses under 16,000 tokens) vastly exceeds GSM8K’s 837.14 tokens, explaining the dramatic difference in response lengths after fine-tuning. These factors substantially influence SelfBudgeter’s final performance, as evidenced by: (1) SelfBudgeter’s accuracy closely mirroring that of its fine-tuned base model, and (2) the response length relationships and matching rate relationships between different SelfBudgeter variants remaining consistent with their respective cold-start models.

5 Discussion

In the Analysis section, we systematically examine SelfBudgeter’s adaptive computation allocation mechanism through four pivotal aspects: its ability to dynamically adjust budgets according to problem complexity, and compliance with token constraints while preserving response quality. This holistic evaluation reveals fundamental insights into how adaptive language models negotiate computational efficiency with task requirements, informing both theoretical understanding and practical deployment considerations.

5.1 Adaptive Budget Allocation

To investigate SelfBudgeter’s capacity for difficulty-aware budget allocation, we conduct empirical evaluations across three mathematical reasoning benchmarks with inherent complexity gradients: GSM8K, MATH, and AIME 2024. Our experimental framework systematically evaluates four architectural variants combining cold-start initialization strategies (GSM8K vs. s1k) with α hyperparameter values (0.2 vs. 0.5).

Figure 4 presents the statistical evidence revealing a consistent positive correlation between problem complexity and allocated token budgets across all architectural variants, demonstrating the system’s fundamental capacity to proportionally scale computational investment with task demands. Notably, the near-linear progression of budget allocation across difficulty tiers validates SelfBudgeter’s emergent capability in intrinsic difficulty quantification. The minimal variance in allocation patterns across configurations underscores the framework’s robust learning of task complexity metrics, regardless of initialization protocols or tradeoff parameter intensity. This consistency in budget-difficulty alignment

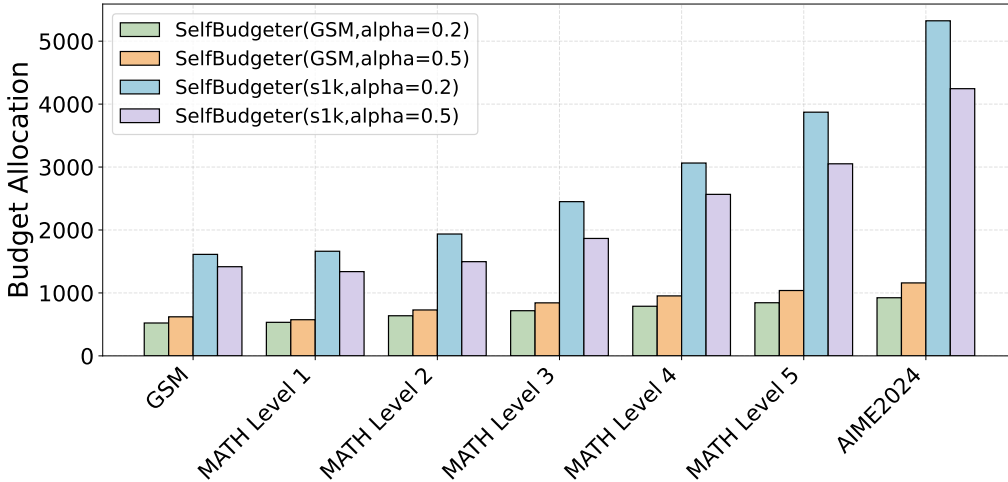


Figure 4: Token budget allocation patterns across problem difficulty levels for four SelfBudgeter configurations (initialized on GSM8K/s1k with $\alpha=0.2/0.5$). All variants exhibit monotonic budget escalation with increasing task complexity (GSM8K, MATH Level 1-5, AIME2024), confirming robust cross-configuration alignment between computational investment and intrinsic problem difficulty.

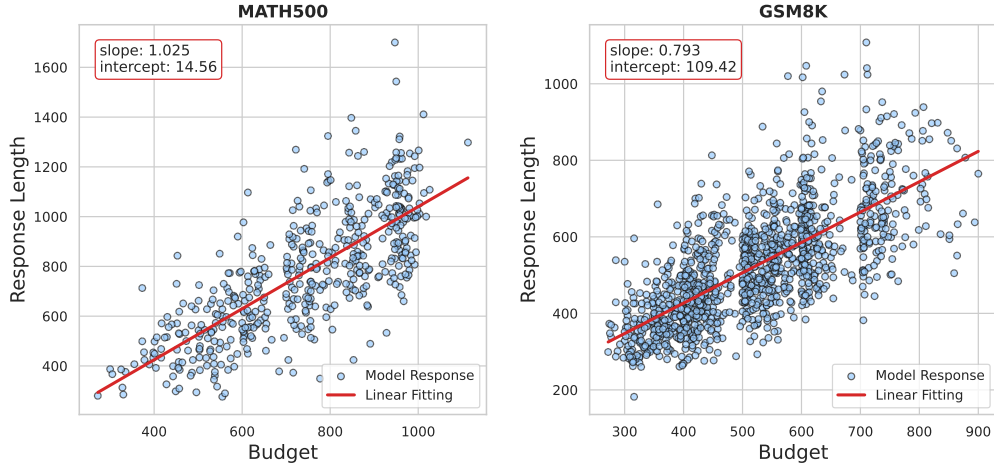


Figure 5: Linear regression analysis of token budget prediction versus actual response length for SelfBudgeter on the MATH500 dataset(left) and GSM8K test set(right). The figure demonstrates SelfBudgeter with GSM initialization and hyperparameter $\alpha = 0.2$. SelfBudgeter effectively generates responses whose lengths align with its predicted token budgets.

suggests the emergence of generalized heuristics for computational resource management, rather than configuration-specific optimization artifacts.

5.2 Budget-Constrained Generation

To systematically evaluate the generation capability of SelfBudgeter under budget constraints, this study employs linear regression modeling to quantitatively analyze the mapping relationship between predicted token budgets and actual response lengths. We conduct a quantitative analysis on the MATH500 dataset and GSM8K test set using linear regression to investigate the mapping between predicted budgets and actual response lengths (as shown in the Figure 5). On MATH500 dataset, the least squares fitting yields a slope of 1.025 (95% CI [0.9466, 1.1042]). And on GSM8K test set, the least squares fitting yields a slope of 0.793 (95% CI [0.7512, 0.8354]). The slope coefficient approaching unity validates the efficacy of the budget control mechanism, indicating that each 1-token increase in the predicted budget corresponds to an average increase of about 1-token in output.

Quantitative results demonstrate that 96% of generated responses exhibit relative deviations $\leq 50\%$ from the target token budget, with 65.40% achieving tighter deviations $\leq 20\%$. Extended experiments on full benchmark datasets reveal that 97.65% (GSM8K) and 95.82% (MATH) of samples satisfy the $\leq 50\%$ relative deviation constraint. Notably, the model’s budget adherence is influenced by the cold-start dataset and hyperparameter α . The optimized SelfBudgeter configuration (initialized with GSM8K and $\alpha = 0.2$), which balances generation quality and budget compliance, is reported here as the best-performing variant.

We further validate SelfBudgeter’s adherence to *user-defined* token budgets through controlled experiments. The results indicate that the actual generated length follows a linear functional relationship with user-defined budgets, demonstrating robust alignment even under explicit external constraints. Details are provided in Appendix B.

6 Conclusion

We propose the SelfBudgeter framework, which autonomously predicts required token budgets for reasoning while effectively adhering to self-imposed constraints, successfully optimizing the accuracy-response length trade-off. By leveraging SelfBudgeter’s token budget predictions, users can anticipate total inference duration in advance, significantly enhancing user experience. In resource-efficient reasoning, SelfBudgeter demonstrates performance comparable to L1 variants, highlighting

its potential for deployment in resource-constrained environments. Additionally, output length can be dynamically regulated through transformation functions when required. SelfBudgeter paves a promising pathway toward more efficient, controllable, and user-friendly reasoning models.

References

- [1] P. Aggarwal and S. Welleck. L1: Controlling how long a reasoning model thinks with reinforcement learning. *arXiv preprint arXiv:2503.04697*, 2025.
- [2] D. Arora and A. Zanette. Training language models to reason efficiently. *CoRR*, abs/2502.04463, 2025. doi: 10.48550/ARXIV.2502.04463. URL <https://doi.org/10.48550/arXiv.2502.04463>.
- [3] S. A. Aytes, J. Baek, and S. J. Hwang. Sketch-of-thought: Efficient llm reasoning with adaptive cognitive-inspired sketching. *arXiv preprint arXiv:2503.05179*, 2025.
- [4] E. Y. Chang, Y. Tong, M. Niu, G. Neubig, and X. Yue. Demystifying long chain-of-thought reasoning in llms. *CoRR*, abs/2502.03373, 2025. doi: 10.48550/ARXIV.2502.03373. URL <https://doi.org/10.48550/arXiv.2502.03373>.
- [5] Q. Chen, L. Qin, J. Liu, D. Peng, J. Guan, P. Wang, M. Hu, Y. Zhou, T. Gao, and W. Che. Towards reasoning era: A survey of long chain-of-thought for reasoning large language models. *arXiv preprint arXiv:2503.09567*, 2025.
- [6] X. Chen, J. Xu, T. Liang, Z. He, J. Pang, D. Yu, L. Song, Q. Liu, M. Zhou, Z. Zhang, R. Wang, Z. Tu, H. Mi, and D. Yu. Do not think that much for $2+3=?$ on the overthinking of o1-like llms, 2024. URL <https://arxiv.org/abs/2412.21187>.
- [7] Z. Chen, Y. Min, B. Zhang, J. Chen, J. Jiang, D. Cheng, W. X. Zhao, Z. Liu, X. Miao, Y. Lu, L. Fang, Z. Wang, and J.-R. Wen. An empirical study on eliciting and improving r1-like reasoning models. *arXiv preprint arXiv:2503.04548*, 2025.
- [8] Y. Chuang, H. Zhou, P. K. Sarma, P. Gopalan, J. Boccio, S. Bolouki, and X. Hu. Learning to route with confidence tokens. *CoRR*, abs/2410.13284, 2024. doi: 10.48550/ARXIV.2410.13284. URL <https://doi.org/10.48550/arXiv.2410.13284>.
- [9] Y.-N. Chuang, L. Yu, G. Wang, L. Zhang, Z. Liu, X. Cai, Y. Sui, V. Braverman, and X. Hu. Confident or seek stronger: Exploring uncertainty-based on-device llm routing from benchmarking to generalization. *arXiv preprint arXiv:2502.04428*, 2025.
- [10] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman. Training verifiers to solve math word problems. *CoRR*, abs/2110.14168, 2021. URL <https://arxiv.org/abs/2110.14168>.
- [11] DeepSeek-AI, D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi, X. Zhang, X. Yu, Y. Wu, Z. F. Wu, Z. Gou, Z. Shao, Z. Li, Z. Gao, A. Liu, B. Xue, B. Wang, B. Wu, B. Feng, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, D. Dai, D. Chen, D. Ji, E. Li, F. Lin, F. Dai, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Bao, H. Xu, H. Wang, H. Ding, H. Xin, H. Gao, H. Qu, H. Li, J. Guo, J. Li, J. Wang, J. Chen, J. Yuan, J. Qiu, J. Li, J. L. Cai, J. Ni, J. Liang, J. Chen, K. Dong, K. Hu, K. Gao, K. Guan, K. Huang, K. Yu, L. Wang, L. Zhang, L. Zhao, L. Wang, L. Zhang, L. Xu, L. Xia, M. Zhang, M. Zhang, M. Tang, M. Li, M. Wang, M. Li, N. Tian, P. Huang, P. Zhang, Q. Wang, Q. Chen, Q. Du, R. Ge, R. Zhang, R. Pan, R. Wang, R. J. Chen, R. L. Jin, R. Chen, S. Lu, S. Zhou, S. Chen, S. Ye, S. Wang, S. Yu, S. Zhou, S. Pan, S. S. Li, S. Zhou, S. Wu, S. Ye, T. Yun, T. Pei, T. Sun, T. Wang, W. Zeng, W. Zhao, W. Liu, W. Liang, W. Gao, W. Yu, W. Zhang, W. L. Xiao, W. An, X. Liu, X. Wang, X. Chen, X. Nie, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yang, X. Li, X. Su, X. Lin, X. Q. Li, X. Jin, X. Shen, X. Chen, X. Sun, X. Wang, X. Song, X. Zhou, X. Wang, X. Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. Zhang, Y. Xu, Y. Li, Y. Zhao, Y. Sun, Y. Wang, Y. Yu, Y. Zhang, Y. Shi, Y. Xiong, Y. He, Y. Piao, Y. Wang, Y. Tan, Y. Ma, Y. Liu, Y. Guo, Y. Ou, Y. Wang, Y. Gong, Y. Zou, Y. He, Y. Xiong, Y. Luo, Y. You, Y. Liu, Y. Zhou, Y. X. Zhu, Y. Xu, Y. Huang, Y. Li, Y. Zheng, Y. Zhu, Y. Ma, Y. Tang, Y. Zha, Y. Yan, Z. Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Xie, Z. Zhang, Z. Hao, Z. Ma, Z. Yan, Z. Wu, Z. Gu, Z. Zhu, Z. Liu, Z. Li, Z. Xie, Z. Song, Z. Pan, Z. Huang, Z. Xu, Z. Zhang, and Z. Zhang. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL <https://arxiv.org/abs/2501.12948>.
- [12] T. Han, Z. Wang, C. Fang, S. Zhao, S. Ma, and Z. Chen. Token-budget-aware llm reasoning. *arXiv preprint arXiv:2412.18547*, 2024.

- [13] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the MATH dataset. In J. Vanschoren and S. Yeung, editors, *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021, December 2021, virtual*, 2021. URL <https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/be83ab3ecd0db773eb2dc1b0a17836a1-Abstract-round2.html>.
- [14] Y. Kang, X. Sun, L. Chen, and W. Zou. C3ot: Generating shorter chain-of-thought without compromising effectiveness. *CoRR*, abs/2412.11664, 2024. doi: 10.48550/ARXIV.2412.11664. URL <https://doi.org/10.48550/arXiv.2412.11664>.
- [15] A. Lee, E. Che, and T. Peng. How well do llms compress their own chain-of-thought? A token complexity approach. *CoRR*, abs/2503.01141, 2025. doi: 10.48550/ARXIV.2503.01141. URL <https://doi.org/10.48550/arXiv.2503.01141>.
- [16] J. LI, E. Beeching, L. Tunstall, B. Lipkin, R. Soletskyi, S. C. Huang, K. Rasul, L. Yu, A. Jiang, Z. Shen, Z. Qin, B. Dong, L. Zhou, Y. Fleureau, G. Lample, and S. Polu. Numinamath. [<https://huggingface.co/AI-M0/NuminaMath-CoT>] (https://github.com/project-numina/aimo-progress-prize/blob/main/report/numina_dataset.pdf), 2024.
- [17] T. Liu, Q. Guo, X. Hu, C. Jiayang, Y. Zhang, X. Qiu, and Z. Zhang. Can language models learn to skip steps? In A. Globersons, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. M. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024. URL http://papers.nips.cc/paper_files/paper/2024/hash/504fa7e518da9d1b53a233ed20a38b46-Abstract-Conference.html.
- [18] H. Luo, L. Shen, H. He, Y. Wang, S. Liu, W. Li, N. Tan, X. Cao, and D. Tao. O1-pruner: Length-harmonizing fine-tuning for o1-like reasoning pruning. *CoRR*, abs/2501.12570, 2025. doi: 10.48550/ARXIV.2501.12570. URL <https://doi.org/10.48550/arXiv.2501.12570>.
- [19] M. Luo, S. Tan, J. Wong, X. Shi, W. Y. Tang, M. Roongta, C. Cai, J. Luo, L. E. Li, R. A. Popa, and I. Stoica. Deepscaler: Surpassing o1-preview with a 1.5b model by scaling rl. <https://pretty-radio-b75.notion.site/DeepScaler-Surpassing-O1-Preview-with-a-1-5B-Model-by-Scaling-RL-19681902c1468005bed8ca303013a4e2>, 2025. Notion Blog.
- [20] X. Ma, G. Wan, R. Yu, G. Fang, and X. Wang. Cot-valve: Length-compressible chain-of-thought tuning. *CoRR*, abs/2502.09601, 2025. doi: 10.48550/ARXIV.2502.09601. URL <https://doi.org/10.48550/arXiv.2502.09601>.
- [21] N. Muennighoff, Z. Yang, W. Shi, X. L. Li, L. Fei-Fei, H. Hajishirzi, L. Zettlemoyer, P. Liang, E. J. Candès, and T. Hashimoto. s1: Simple test-time scaling. *CoRR*, abs/2501.19393, 2025. doi: 10.48550/ARXIV.2501.19393. URL <https://doi.org/10.48550/arXiv.2501.19393>.
- [22] T. Munkhbat, N. Ho, S. H. Kim, Y. Yang, Y. Kim, and S. Yun. Self-training elicits concise reasoning in large language models. *CoRR*, abs/2502.20122, 2025. doi: 10.48550/ARXIV.2502.20122. URL <https://doi.org/10.48550/arXiv.2502.20122>.
- [23] S. Nayab, G. Rossolini, G. C. Buttazzo, N. Manes, and F. Giacomelli. Concise thoughts: Impact of output length on LLM reasoning and cost. *CoRR*, abs/2407.19825, 2024. doi: 10.48550/ARXIV.2407.19825. URL <https://doi.org/10.48550/arXiv.2407.19825>.
- [24] I. Ong, A. Almahairi, V. Wu, W. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica. Routellm: Learning to route llms with preference data. *CoRR*, abs/2406.18665, 2024. doi: 10.48550/ARXIV.2406.18665. URL <https://doi.org/10.48550/arXiv.2406.18665>.
- [25] OpenAI. Learning to reason with llms, September 2024. URL <https://openai.com/index/learning-to-reason-with-llms/>.
- [26] R. Pan, Y. Dai, Z. Zhang, G. Oliaro, Z. Jia, and R. Netravali. Specreason: Fast and accurate inference-time compute via speculative reasoning, 2025. URL <https://arxiv.org/abs/2504.07891>.
- [27] Qwen. Qwq: Reflect deeply on the boundaries of the unknown, November 2024. URL <https://qwenlm.github.io/blog/qwq-32b-preview/>.

- [28] Qwen, :, A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, H. Lin, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Lin, K. Dang, K. Lu, K. Bao, K. Yang, L. Yu, M. Li, M. Xue, P. Zhang, Q. Zhu, R. Men, R. Lin, T. Li, T. Tang, T. Xia, X. Ren, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Wan, Y. Liu, Z. Cui, Z. Zhang, and Z. Qiu. Qwen2.5 technical report, 2025. URL <https://arxiv.org/abs/2412.15115>.
- [29] M. Renze and E. Guven. The benefits of a concise chain of thought on problem-solving in large language models. In *2nd International Conference on Foundation and Large Language Models, FLLM 2024, Dubai, United Arab Emirates, November 26-29, 2024*, pages 476–483. IEEE, 2024. doi: 10.1109/FLLM63129.2024.10852493. URL <https://doi.org/10.1109/FLLM63129.2024.10852493>.
- [30] C. Snell, J. Lee, K. Xu, and A. Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *CoRR*, abs/2408.03314, 2024. doi: 10.48550/ARXIV.2408.03314. URL <https://doi.org/10.48550/arXiv.2408.03314>.
- [31] Y. Sui, Y.-N. Chuang, G. Wang, J. Zhang, T. Zhang, J. Yuan, H. Liu, A. Wen, H. Chen, X. Hu, et al. Stop overthinking: A survey on efficient reasoning for large language models. *arXiv preprint arXiv:2503.16419*, 2025.
- [32] H. Veeraboina. Aime problem set 1983-2024, 2023. URL <https://www.kaggle.com/datasets/hemishveeraboina/aime-problem-set-1983-2024>.
- [33] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, 2022. URL http://papers.nips.cc/paper_files/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html.
- [34] Y. Wu, Y. Wang, T. Du, S. Jegelka, and Y. Wang. When more is less: Understanding chain-of-thought length in llms. *CoRR*, abs/2502.07266, 2025. doi: 10.48550/ARXIV.2502.07266. URL <https://doi.org/10.48550/arXiv.2502.07266>.
- [35] H. Xia, Y. Li, C. T. Leong, W. Wang, and W. Li. Tokenskip: Controllable chain-of-thought compression in llms. *CoRR*, abs/2502.12067, 2025. doi: 10.48550/ARXIV.2502.12067. URL <https://doi.org/10.48550/arXiv.2502.12067>.
- [36] S. Xu, W. Xie, L. Zhao, and P. He. Chain of draft: Thinking faster by writing less. *CoRR*, abs/2502.18600, 2025. doi: 10.48550/ARXIV.2502.18600. URL <https://doi.org/10.48550/arXiv.2502.18600>.
- [37] Y. Xu, H. Dong, L. Wang, D. Sahoo, J. Li, and C. Xiong. Scalable chain of thoughts via elastic reasoning, 2025. URL <https://arxiv.org/abs/2505.05315>.
- [38] W. Yang, S. Ma, Y. Lin, and F. Wei. Towards thinking-optimal scaling of test-time compute for llm reasoning. *arXiv preprint arXiv:2502.18080*, 2025.

A Training Details

A.1 Experimental Environments

Our server is equipped with two 80GB A100 GPUs and two 45GB A40 GPUs. We conducted fine-tuning experiments and inference tests on the two A40 GPUs, while the GRPO training was performed on the two A100 GPUs.

A.2 Parameter Settings

In the fine-tuning training during the cold-start phase, our parameter settings are configured as follows. The sequence length is capped at 16,384, with a per-device training and evaluation batch size of 1, while gradient accumulation (2 steps) is employed to alleviate GPU memory constraints. A cosine learning rate scheduler is adopted with a 10% warm-up ratio and a base learning rate of $5e-5$. The model is trained for 1 epoch, with 10% of the training set allocated for validation. The model checkpoints are saved and evaluated every 500 steps, and the best-performing checkpoint is retained.

In the GRPO (Global Reward Policy Optimization) training, our parameter configuration is set as follows. The training and validation batch sizes are set to 128 and 1,250, respectively, with maximum prompt and response lengths of 1,024 and 4,096 tokens. The Actor model employs a learning rate of $1e-6$, dynamic batching (up to 24K tokens per GPU), and a KL divergence loss (coefficient 0.001), with gradient checkpointing and FSDP (Fully Sharded Data Parallel) distributed training enabled (parameter offloading disabled). During the Rollout phase, the vLLM inference engine is utilized with tensor parallelism (TP=2) and 80% GPU memory utilization, generating 5 responses per round. Global settings include 3 training epochs, a checkpoint-saving interval of 50 steps, and a KL control coefficient of 0.001, executed on a single node with dual GPUs. And key hyperparameters involved in the reward function are specified in Table 3.

Table 3: Hyperparameters Settings

Parameters	$C = 0$	$C = 1$	Parameters	Value
$s_{\min}$	-0.5	0.5	r_f	-1
$s_{\max}$	0	1	r_b	-0.4

For the GSM-initialized SelfBudgeter, we select the checkpoint after 699 training steps when alpha was set to 0.2, and the checkpoint after 575 steps when alpha was 0.5. For the slk-initialized SelfBudgeter, we choose the checkpoint after 475 training steps with alpha=0.2, and the checkpoint after 500 steps with alpha=0.5. For L1-Max, we choose the checkpoint after 280 training steps.

B Prefilled Token Budget Following

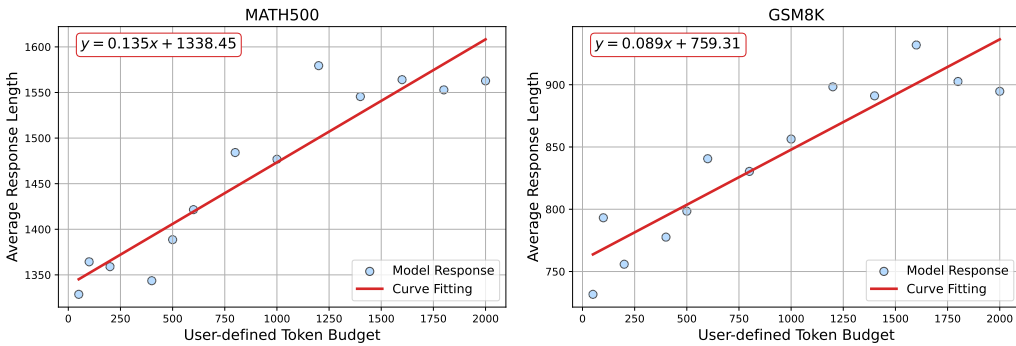


Figure 6: Relationship between user-defined token budgets and SelfBudgeter average response lengths with curve fitting analysis.

To systematically evaluate model performance under user-defined token budget constraints, we conduct quantitative analysis using SelfBudgeter with GSM initialization and hyperparameter $\alpha = 0.2$ on both MATH500 dataset and GSM8K test set. In the experimental design, fixed token budgets were pre-filled in the <budget> field of training templates, with empirical results obtained by measuring average generated response lengths. We evaluated SelfBudgeter’s performance with user-defined token budgets ranging from 50 to 2000 (specifically: 50, 100, 200, 400, 500, 600, 800, 1000, 1200, 1400, 1600, 1800, and 2000), as shown in the Figure 6.

Regression intercepts effectively reflect problem complexity, where GSM8K’s simpler questions yield significantly smaller intercepts. Despite a moderate slope, SelfBudgeter demonstrates robust budget adaptability, maintaining a stable positive correlation between user-defined budgets and output lengths. This linear relationship enables deterministic length control through derived transformation functions.

C Limitations

While this study presents promising results, it is not without certain inherent limitations that warrant consideration. First and foremost, the proposed methodology, though theoretically sound, may face practical challenges when applied to more complex or diverse real-world scenarios, as is often the case with cutting-edge research. Additionally, while extensive efforts have been made to ensure the robustness of our findings, certain external factors beyond the scope of this work could potentially influence the generalizability of the results. These limitations, however, do not detract from the overall contributions of this work but rather highlight opportunities for future research directions.

D Broader Impacts

Our research involves extensive training and testing of large language models (LLMs), a computationally intensive process that consumes substantial electricity and contributes to significant carbon emissions—an important environmental concern in AI development.

However, the primary societal benefit of this work lies in its potential to substantially optimize LLM inference efficiency, thereby reducing long-term computational resource demands. By developing more resource-efficient inference methods, our approach could help mitigate future energy consumption and carbon footprint during model deployment, contributing to sustainable AI practices. This trade-off between short-term environmental costs and long-term efficiency gains reflects a critical consideration in responsible AI research, where initial energy investments may yield disproportionate reductions in operational impacts over time. The net societal impact thus depends on achieving sufficient inference efficiency improvements to offset the upfront training costs—a key challenge we address through our methodological innovations.